SAVEETHA SCHOOL OF ENGINEERING

SIMATS Engineering

Department of Computer Science and Engineering

OPERATING SYSTEMS

ASSIGNMENT 1

CO1: Apply the concepts of operating system types, structures and system calls to demonstrate their functions in various computing environments – history of operating systems, mainframe, desktop, multiprocessor, distributed, clustered, real-time and handheld systems, OS structure, services, system calls, system design, CPU states, memory hierarchy and I/O systems.

Assignment Title

Evaluate and Justify the Operating-System Design of a Smart Hospital

z(Total Marks – 50)

Name	Hakesh C
Register Number	1972260066
Course Code	CSA0403
Course Name	Operating Systems
Assignment	Assignment 1

Scenario – Operating Systems for a Smart Hospital

A 500-bed hospital is replacing its computer systems. It needs bedside patient monitors that raise an alarm within 10 ms of an abnormal reading, nurses' handheld tablets, doctors' desktop workstations, a central patient-record database server that must never stop, an imaging server that processes MRI scans on many processors, and a link to two partner hospitals. The IT team proposes one general-purpose OS everywhere; the clinical staff fear delayed alarms, database stoppage during updates and short tablet battery life. The system may grow from 1 ward (40 monitors, 10 tablets) to 1 hospital (500 monitors, 120 tablets, 80 workstations) to a group of 3 hospitals.

TASK 1 – Evaluate the Requirements of Each Computing Environment

Task 1 (K5 – Evaluate) **[4 Marks]**

Evaluate what each part of the hospital needs from its operating system. Discuss:

- Response-time, reliability, throughput, battery and security requirements of each device
- Which requirements conflict, and which are critical to patient safety
- How lessons from the history of operating systems (batch, time-sharing, personal, mobile) apply

Rank the requirements of each environment and justify the ranking.

Alternative Concept: If the whole hospital used a single general-purpose OS on every device, evaluate the worst-case alarm response time (unbounded delay vs guaranteed deadline), memory needed per device and administration effort.

Answer:

1.1 Ranked requirement table (1 = most important)

Environment	Rank 1	Rank 2	Rank 3	Rank 4	Rank 5	Basis
Bedside monitor	Response time (10 ms)	Reliability	Security	Battery (backup only)	Throughput	Safety deadline
Nurse tablet	Battery	Security	Response time	Reliability	Throughput	Mobile, lost-device risk
Doctor workstation	Response time (interactive)	Security	Throughput	Reliability	Battery (n/a)	Interactive use
Database server	Reliability (24x7)	Security	Throughput	Response time	Battery (n/a)	Never stops
Imaging server	Throughput	Reliability	Response time	Security	Battery (n/a)	Heavy computing
Partner-hospital link	Security	Reliability	Throughput	Response time	Battery (n/a)	Records leave site

Justification. The monitor is ranked response-first because a late alarm is a clinical failure, not an inconvenience; throughput is last because it only samples a few sensors. The tablet runs on a battery for a full shift and can be lost, so battery and security lead. The workstation is a human-interactive machine, so a fast response matters most. The database holds the legal record of every patient and must be available continuously, so reliability dominates; throughput matters next because many clients query it. The imaging server is a batch-style, compute-bound machine and values the number of scans processed per hour above latency. The partner link

carries confidential records between organisations, so confidentiality and integrity (security) are ranked first.

1.2 Conflicting requirements

- **Fast response vs many users on one machine:** time-sharing among many users adds scheduling delay, so a machine serving many users cannot also guarantee a 10 ms deadline.
- **Throughput vs response time:** large time slices and batching raise throughput but delay interactive and alarm tasks.
- **Battery vs performance:** faster CPU clocks and constant polling drain the tablet and the monitor's backup battery.
- **Security vs response time/ease of use:** encryption, authentication and auditing add CPU and delay; frequent log-ins slow the clinical staff.
- **Reliability vs updating:** security patches usually need reboots, which conflicts with a database that must never stop.

1.3 Requirements critical to patient safety

(i) The 10 ms alarm deadline on the monitors (hard real-time); (ii) continuous availability and data integrity of the patient-record database; (iii) protection of one patient's data from another user (privacy and correct identification); (iv) reliable, authentic data in the partner link. Throughput of the imaging server and tablet battery are important for efficiency but are not immediately life-critical.

1.4 Lessons from the history of operating systems

Generation	Lesson	Application in the hospital
Batch (1950s–60s)	Keep the CPU busy; process jobs in groups without user interaction	Imaging server: MRI reconstruction jobs queued and run for maximum throughput
Time-sharing (1960s–70s)	Share the CPU quickly among users for good response; needs protection and memory management	Database server and workstations: many clinicians served at once with fair scheduling and file protection
Personal (1980s–)	One user, convenience and a graphical interface; little protection originally	Doctor workstations: responsiveness and usability, but must add login security
Mobile/handheld (2000s–)	Power management, touch interface, small memory, apps sandboxed from each other	Nurse tablets: sandboxed apps, battery saving, remote wipe
Real-time (embedded)	Deadlines are guaranteed by design, not by speed	Bedside monitors: priority-based pre-emptive scheduling with bounded latency

1.5 Alternative concept: one general-purpose OS everywhere

Criterion	Single general-purpose OS (e.g. desktop Linux/Windows)	Specialised OS per environment
Worst-case alarm response	Unbounded: a page fault, driver, garbage collection or a higher-priority load can delay the alarm by tens to hundreds of ms – the 10 ms deadline cannot be guaranteed	Bounded: an RTOS pre-empts in microseconds; the worst case is known and below 10 ms
Memory per device	About 1–4 GB RAM and many GB of disk on every device, including monitors	Monitor RTOS: kilobytes to a few MB; tablet OS: 1–4 GB; only servers need large memory

Criterion	Single general-purpose OS (e.g. desktop Linux/Windows)	Specialised OS per environment
Administration effort	Low – one image, one tool, one skill set (+)	Higher – several OS families and update procedures (–)
Patch/ reboot risk	Same patch cycle reboots monitors and database	Each environment patched independently

Evaluation: the single-OS idea reduces administration but fails the safety requirement because the worst-case latency is unbounded (average latency is irrelevant for an alarm). It also wastes memory and battery. Therefore a different OS type per environment is justified, accepting the extra administration cost.

TASK 2 – Judge the Type of Operating System for Each Environment

> **Task 2 (K5 – Evaluate)** **[3 Marks]**
>
> Compare mainframe, desktop, multiprocessor, distributed, clustered, real-time (hard/soft) and handheld systems; choose the best type for monitors, tablets, workstations, database server, imaging server and partner link; discuss symmetric vs asymmetric multiprocessing for imaging and a clustered fail-over for the database.
>
> **Alternative Concept:** If the database used a single large multiprocessor instead of a fail-over cluster, evaluate availability, cost and how throughput grows with processors (time) and memory (space).

Answer:

2.1 Types of operating systems compared

Type	Main characteristic	Suitability for the hospital
Mainframe	Batch/transaction oriented, very high I/O and reliability	Possible but costly for records; over-sized for a 500-bed hospital
Desktop	Single user, GUI, good response, modest protection	Doctor workstations
Multiprocessor (parallel)	Several CPUs share memory and bus; more throughput and reliability	Imaging server
Distributed	Independent computers communicate over a network and appear as one system	Partner-hospital link and shared records
Clustered	Nodes share storage, monitor each other and fail over	Patient-record database
Hard real-time	Missing a deadline is a failure; guaranteed bounded latency	Bedside monitors (alarm in 10 ms)
Soft real-time	Deadlines are important but an occasional miss is tolerated	Video/audio, tablet alerts
Handheld	Small memory/screen, battery, touch, sandboxed apps	Nurse tablets

2.2 Chosen type for each environment, with rejected alternatives

Environment	Chosen type	Rejected alternatives and reasons
Bedside monitor	Hard real-time OS (e.g. QNX/FreeRTOS)	Desktop/handheld OS: no deadline guarantee; soft real-time: a missed alarm is unacceptable

Environment	Chosen type	Rejected alternatives and reasons
Nurse tablet	Handheld OS (Android/iOS-like, soft real-time)	Desktop OS: heavy, short battery life, not touch-oriented
Doctor workstation	Desktop OS (time-sharing, GUI)	Handheld: small screen; mainframe terminal: loss of local productivity
Database server	Clustered server OS with fail-over	Single machine: single point of failure; mainframe: high cost
Imaging server	Symmetric multiprocessing OS	Single processor: too slow; asymmetric: master CPU bottleneck
Partner link	Distributed OS services (secure network, authenticated RPC/replication)	Manual file transfer: slow, error-prone, insecure

2.3 Symmetric vs asymmetric multiprocessing (imaging server)

In **asymmetric multiprocessing (AMP)** one master CPU runs the OS and assigns work to slave CPUs; it is simple but the master becomes a bottleneck and a single point of failure. In **symmetric multiprocessing (SMP)** every CPU runs the same OS and may run kernel or user code; any CPU can take any MRI task, the load is balanced automatically, and the failure of one CPU only reduces capacity. The cost is more complex kernel locking and cache-coherence overhead. Because MRI processing splits naturally into many independent slices, **SMP is the better choice**.

2.2.1 Clustered system with fail-over (database server)

Two or more database nodes share storage; a heartbeat message is exchanged every second. When the active node stops, the standby detects the missing heartbeat, takes over the shared disk and the service IP address within seconds, and clients reconnect. Updates are applied to one node at a time (rolling update), so the service never stops during patching – exactly the clinical staff's concern.

2.4 Alternative concept: one large multiprocessor instead of a cluster

Criterion	Single large multiprocessor	Cluster with fail-over
Availability	Single point of failure: a power, board or OS failure stops everything. For example 99.9% availability = about 8.8 h downtime/year	Two independent nodes of 99.9% each: about 99.9999% (under 1 minute/year) plus 10–30 s fail-over
Cost	Cost grows faster than capacity (specialised hardware); one machine must be replaced entirely	Commodity nodes; add nodes as hospitals join; needs shared storage and licences
Throughput vs processors (time)	Limited by Amdahl's law: speed-up = 1 / ((1 – p) + p / N). With p = 0.9, 16 CPUs give only 6.4x	Scale-out by adding nodes, but needs data partitioning/replication; writes remain serialised
Memory (space)	All memory is shared and directly addressable by every CPU; very large RAM possible	Memory is per node; data are replicated or partitioned, so more total RAM is used

Conclusion: the cluster meets the "never stop" requirement; the single multiprocessor is simpler to program but is rejected because of its single point of failure.

TASK 3 – Evaluate Operating-System Structures

Task 3 (K5 – Evaluate) **[4 Marks]**

Evaluate simple (monolithic), layered, microkernel, modular and hybrid structures; their effect on reliability, performance and ease of updating; which structure suits the monitors and the database; how Linux, Windows, Android and FreeRTOS/QNX are structured.

Alternative Concept: If bedside monitors used a monolithic kernel instead of a microkernel, evaluate the cost of a service request (system call vs message passing), kernel memory and the effect of a faulty driver.

Answer:

3.1 Comparison of structures

Structure	Description	Reliability	Performance	Ease of updating
Simple / monolithic	All OS services in one large kernel address space	Low – a bug anywhere can crash the kernel	Highest – direct function calls	Hard – rebuild/reboot
Layered	OS divided into layers; each uses only the layer below	Good – easy to debug layer by layer	Lower – each request crosses layers	Moderate
Microkernel	Only scheduling, IPC and memory in the kernel; drivers, file systems are user-space servers	Highest – a server can crash and restart	Lower – message passing and context switches	Easy – restart a single server
Modular (loadable modules)	Small core kernel plus modules loaded at run time	Moderate – module runs in kernel mode	High – direct calls	Easy – load/unload a module
Hybrid	Microkernel design with some services kept in kernel mode	Good	Good	Moderate

3.2 Structures of real systems

- **Linux:** monolithic kernel with loadable kernel modules (modular).
- **Windows (NT):** hybrid – a microkernel-style executive with many services (including graphics and drivers) running in kernel mode.
- **Android:** Linux kernel at the bottom, then hardware abstraction layer, native libraries and the Android runtime, application framework and apps (layered).
- **FreeRTOS:** a very small kernel (scheduler, queues, semaphores, timers) linked with the application; **QNX:** true microkernel with drivers and file systems as user-space processes communicating by messages.

3.3 Choice for two environments

Bedside monitor – microkernel RTOS (QNX-type). Reliability and bounded latency matter more than raw speed. If the display or network driver fails, only that process is restarted while the alarm task keeps running, and the small kernel is easier to verify and certify for medical use. **Database server – monolithic kernel with loadable modules (Linux).** Maximum I/O and network throughput is needed and the hardware is fixed; the cluster and the mature file-system code provide the reliability, and modules allow updating drivers without rebuilding the kernel.

3.4 Alternative concept: monolithic kernel on the monitors

Criterion	Monolithic kernel	Microkernel
Time per service request	One system call: about 0.1–1 µs (trap + direct function call)	Message passing to a server: two or more context switches, typically 2–10x slower (a few µs)
Kernel memory	Larger: tens of MB with all drivers and file systems built in	Tiny kernel (tens to hundreds of KB); servers are separate processes
Faulty driver	Runs in kernel mode and can corrupt the kernel – the whole monitor crashes and no alarm is raised	Driver is a user process; the fault is contained and the server is restarted

Evaluation: the few extra microseconds of IPC are far smaller than the 10 ms alarm deadline, so the performance penalty is acceptable, while containing a faulty driver is a patient-safety benefit. The microkernel is therefore justified.

TASK 4 – Evaluate the Services an Operating System Must Provide

> **Task 4 (K5 – Evaluate)** **[4 Marks]**
>
> Evaluate the services each environment needs: user interface, program execution, I/O operations, file systems, communication, error detection, resource allocation, accounting, protection and security. Which services can be removed from the bedside monitor, which are essential for the database server, and how the OS protects one patient's records from another user's programs.
>
> **Alternative Concept:** If the tablets used a full desktop OS with every service enabled instead of a reduced handheld OS, evaluate memory/storage (space), start-up time and battery life (time).

Answer:

4.1 Service table (E = essential, R = reduced, X = excluded)

Service	Monitor	Tablet	Work-station	DB server	Imaging	Partner link
User interface	X (small display)	E (touch)	E (GUI)	R (admin console)	R	X
Program execution	E (fixed tasks)	E	E	E	E	E
I/O operations	E (sensors, alarm)	E	E	E (disk, network)	E (DMA)	E (network)
File system	R (small/none)	E	E	E (journalled)	E	R
Communication	E (to nurse station)	E (Wi-Fi)	E	E	E	E (secure)
Error detection	E	E	E	E (ECC, logs)	E	E
Resource allocation	R (static)	E	E	E	E (CPUs, memory)	E
Accounting / audit	X	R	R	E (access log)	R	E
Protection & security	R	E (encryption)	E	E	E	E (authentication)

4.2 Services removed from the bedside monitor

The graphical desktop and window system, general-purpose multi-user login, printing, a large general-purpose file system, user accounting, and dynamic loading of arbitrary programs can

be removed. Only the scheduler, interrupt handling, sensor and alarm drivers, a small network stack and basic error detection remain. A smaller image boots faster, has fewer bugs and less attack surface, and leaves predictable timing.

4.3 Services essential for the database server

Reliable and journalled file system, process/thread management for many concurrent clients, memory management with caching, fast disk and network I/O, error detection and recovery (ECC memory, disk RAID, logs), resource allocation, authentication, access control, encryption, accounting/audit trails and clustering/fail-over communication.

4.4 How records of one patient are protected from other users' programs

- **Authentication:** each user logs in with a password/smart card and receives a user ID.
- **Authorisation:** file permissions or access-control lists and database roles allow only the treating team to read a patient's record.
- **Memory protection:** each process has its own address space; the MMU prevents a program from reading another program's memory.
- **Kernel mode:** only the OS can access the disk; programs must use system calls, which check permissions.
- **Encryption and audit:** data are encrypted on disk and in transit, and every access is logged for accounting.

4.5 Alternative concept: full desktop OS on tablets

Criterion	Full desktop OS	Reduced handheld OS
Memory/storage (space)	About 4–8 GB RAM and 20–40 GB of storage	About 1–4 GB RAM and 4–10 GB of storage
Start-up time	30–60 s	5–15 s
Battery life	Background services and a larger CPU load give about 3–5 h	About 8–12 h, i.e. a full shift

(Typical figures; they depend on the hardware.) The extra services (printing, desktop shell, many daemons) are not needed by a nurse and would shorten battery life below one shift, so the reduced handheld OS is chosen.

TASK 5 – Examine System Calls

> **Task 5 (K5 – Evaluate)** **[4 Marks]**
>
> Examine how a program obtains OS services. Discuss the categories of system calls, what happens step by step when a program calls read(), how parameters are passed to the kernel, and how strace shows the system calls.
>
> **Alternative Concept:** If programs called fopen, fread and fwrite instead of open, read and write directly, evaluate the number of system calls for a file of n bytes (about n/4096 with library buffering vs n/b for a buffer of b bytes) and the extra buffer space.

Answer:

5.1 Categories of system calls

Category	Purpose	Examples (Linux)
Process control	Create, end, load, wait for processes	fork, execve, wait, exit, kill

Category	Purpose	Examples (Linux)
File management	Create, open, read, write, delete files	open, read, write, close, unlink
Device management	Request/release devices, read/write them	ioctl, read, write, mmap
Information maintenance	Get/set time, process and system data	getpid, time, uname, sysinfo
Communication	Message passing, pipes, shared memory, sockets	pipe, socket, send, recv, shmget
Protection	Control access to resources	chmod, chown, umask, setuid

5.2 What happens when a program calls read(fd, buf, n)

- **Library call:** the program calls the C-library wrapper read(); the wrapper places the system-call number (0 on x86-64 Linux) and the arguments in registers.
- **Trap:** the wrapper executes a special instruction (syscall/int 0x80), a software interrupt that transfers control to the kernel's system-call entry point.
- **Mode switch:** the CPU changes from user mode to kernel mode (mode bit 1) and switches to the kernel stack.
- **Kernel work:** the dispatcher uses the number as an index into the system-call table, checks the arguments (valid descriptor, valid user buffer, permissions), finds the file in the buffer cache or starts a disk read (the process may block and another process runs), then copies data to the user buffer.
- **Return:** the kernel stores the return value (bytes read, 0 at end of file, or –1 with errno) in a register, switches back to user mode and the wrapper returns to the program.

5.3 Parameter passing

Method	How it works	Used by
Registers	Parameters placed in CPU registers (up to about 6)	Linux x86-64 (rdi, rsi, rdx, r10, r8, r9)
Block (table) in memory	Parameters stored in a block; its address is passed in a register	Linux when there are many parameters, Solaris
Stack	Parameters are pushed on the stack by the program and popped by the OS	Older Linux x86 (32-bit) conventions, some systems

Registers are fastest; the block and stack methods do not limit the number or length of parameters.

5.4 Evidence – strace of a real program

Command used: **strace -e trace=openat,read,write,close cat p.txt** (p.txt contains "hello hospital"). Output obtained on the test system:

```
openat(AT_FDCWD, "p.txt", O_RDONLY)      = 3
read(3, "hello hospital\n", 131072)      = 15
write(1, "hello hospital\n", 15)         = 15
read(3, "", 131072)                      = 0
close(3)                                 = 0
close(1)                                 = 0
close(2)                                 = 0
+++ exited with 0 +++
```

Reading the trace: openat returns file descriptor 3; read asks for 131072 bytes and gets 15; write sends them to descriptor 1 (the screen); the second read returns 0, which means end of

file; the descriptors are closed. strace therefore shows each system call with its arguments and return value, and strace -c gives a summary table of counts and times.

5.5 Alternative concept: fopen/fread/fwrite (library buffering)

Criterion	Library calls (fopen, fread, fwrite)	Direct system calls with a buffer of b bytes
Number of system calls for n bytes	About n / 4096 (the library reads a 4 KB block internally even if the program asks for 1 byte)	n / b; for b = 1 it is n calls
Time complexity	O(n / 4096) = O(n) but with a small constant	O(n / b), best when b is large
Extra buffer space	One buffer per open FILE (4 KB–8 KB) held in user space: O(1)	Buffer chosen by the programmer: b bytes

For n = 10,000 bytes the library needs about 3 system calls, whereas one-byte read() calls need 10,001. The library therefore protects naive programs from the high cost of system calls, at the price of a few KB of memory per file, and the portable C interface; direct calls are best when the programmer controls the buffer size (as in the backup program).

TASK 6 – Evaluate the Cost of System Calls

Task 6 (K5 – Evaluate) **[4 Marks]**

Evaluate how the number of system calls affects performance: why a system call costs much more than a function call; how buffer size changes the number of read() and write() calls; how library buffering reduces system calls; the best buffer size for the hospital's backup program, using measurements.

Complete the Code: Complete TODO 1 using read() and write(), execute on Linux, and compare with strace -c.

Alternative Concept: If the backup used mmap instead of read()/write(), evaluate the number of system calls (a few vs about n/b), copying between kernel and user space, and memory for very large files.

Answer:

6.1 Why a system call costs more than a function call

An ordinary function call is a jump and a return (a few nanoseconds). A system call requires a trap, a mode switch to kernel mode, saving and restoring registers, switching stacks, validating the parameters, running the kernel code (possibly blocking for I/O) and returning; kernel entry also disturbs the CPU caches and the TLB and may involve security mitigations. A system call typically costs hundreds of nanoseconds to microseconds – about 10 to 100 times a function call – and reading one byte per call multiplies this cost by n.

6.2 Completed code (TODO 1 replaced by a read/write loop)

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    int in = open(src, O_RDONLY);
    if (in < 0) return -1;
```

```
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }
    long calls = 0;
    ssize_t n;
    while (1) {
        n = read(in, buf, bufsize);
        calls++;
        if (n <= 0) break;
        if (write(out, buf, n) != n) break;
    }
    close(in);
    close(out);
    return calls;
}
```

The loop reads up to bufsize bytes, counts every call (including the final call that returns 0), stops when read returns 0 or –1 and writes exactly the n bytes that were read.

Compile and run: **gcc -std=c11 -Wall -Wextra t6.c -o t6 && ./t6**

Output obtained (same as the expected output):

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

Check: the number of reads is ceil(10000 / b) + 1, i.e. 10000 + 1, 20 + 1 and 3 + 1; the extra 1 is the final read that returns 0 (end of file).

6.3 Comparison with strace -c

Command: **strace -c ./t6**. Result (extract):

```
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 52.03    0.011800           1     10027           write
 46.72    0.010595           1     10027           read
 0.59    0.000134          14         9           close
 0.40    0.000091          10         9           openat
```

strace reports 10,027 read() calls. The program printed 10,001 + 21 + 4 = 10,026; the one extra read is made by the dynamic loader when it loads libc at start-up. The difference is therefore explained and the two counts agree.

6.4 Measurements for the backup program (copying a 4 MB file)

Buffer size	read() calls	Real time (s)	User time (s)	Kernel (sys) time (s)
1 byte	4,194,304	2.473	0.517	1.786
64 bytes	65,536	0.037	–	–
512 bytes	8,192	0.014	0.000	0.008
4096 bytes	1,024	0.007	0.000	0.004
65536 bytes	64	0.007	0.000	0.003

(Measured on Ubuntu 24.04, gcc 13.3. Real time comes from a timing program; user and sys times come from the shell time command. Values differ slightly between runs and machines, so repeat the test on your own system.)

Library buffering: fread/fwrite keep an internal buffer (4–8 KB), so 10,000 fputc() calls produced only 3 write() calls in the program that creates the test file. **Best buffer size:** time falls rapidly up to 4 KB (the page size and file-system block size) and there is almost no further gain after 4–64 KB, while larger buffers use more memory. For the hospital's backup program

a buffer of **64 KB** (a multiple of the block size) is recommended: it makes 64 calls per 4 MB, gives the best time and uses negligible memory.

6.5 Alternative concept: memory-mapped files (mmap)

Criterion	mmap	read()/write() with buffer b
System calls	A few: open, mmap, (madvise), munmap, close – O(1)	About n / b read calls and n / b write calls – O(n / b)
Copying kernel ↔ user	None – the file's page-cache pages are mapped into the process; pages are loaded on demand by page faults	One copy per read/write between the kernel buffer cache and the user buffer
Memory for very large files	Needs virtual address space for the whole file (limited on 32-bit) and page faults replace system calls; resident memory is managed by the OS	Constant: only the buffer (b bytes)

mmap is attractive for random access and for large read-mostly files, but for a sequential backup the page-fault cost and possible I/O errors raised as signals (SIGBUS) remove most of the advantage; a 64 KB buffered copy is simpler and equally fast.

TASK 7 – Evaluate Process Control System Calls

Task 7 (K5 – Evaluate) **[4 Marks]**

Evaluate how a command shell on the database server starts and manages programs: what fork(), exec(), wait() and exit() do; how a shell runs a command and obtains its exit status; zombie and orphan processes; why fork()+exec() is used instead of a single "create process" call.

Complete the Code: Complete TODO 1 with fork(), execlp() and waitpid(). Explain why stdout is flushed before each fork(); remove the second fflush, change _exit(3) to exit(3), redirect to a file and explain why "shell: running job 2" appears twice.

Alternative Concept: If commands were run with posix_spawn() or threads instead of fork+exec, evaluate the time to start 1,000 commands, memory copied/shared (copy-on-write) and isolation.

Answer:

7.1 The four process-control system calls

Call	Function
fork()	Creates a new child process that is a copy of the parent (same code, data, open files). Returns the child's PID to the parent and 0 to the child.
exec()	Replaces the current process image (code, data, stack) with a new program; the PID and open files remain. It does not return unless it fails.
wait()/waitpid()	Blocks the parent until a child terminates and returns the child's exit status; it also releases the child's process-table entry.
exit()	Terminates the process, closes its files, frees its memory and passes an exit status (0 = success) to the parent. _exit() ends at once without flushing stdio buffers.

7.2 How a shell runs a command and gets its exit status

The shell reads the command line, calls fork(); the child calls exec() with the program name and arguments (after setting up redirection and pipes); the parent calls waitpid() and sleeps. When the program ends it calls exit(status); the kernel wakes the parent, and waitpid() returns the status, which the shell extracts with WEXITSTATUS and stores in $?. For a background job (&) the shell does not wait.

7.3 Zombie and orphan processes

- **Zombie:** a child that has terminated but whose parent has not yet called wait(); only its process-table entry (PID and exit status) remains. The OS removes it when the parent calls wait(); if the parent exits, init/systemd adopts and reaps it. Many zombies can exhaust the process table.
- **Orphan:** a child whose parent has terminated while the child is still running. The kernel re-parents it to init/systemd (PID 1), which calls wait() when the orphan finishes, so no zombie is left.

7.4 Why fork() followed by exec()

Separating creation and loading lets the child change its own environment between the two calls – redirect input/output, set up pipes, change directory, set environment variables, drop privileges – before the new program runs, using ordinary system calls. A single "create process" call would need a very large list of parameters for all of these options. The cost of copying is small because fork uses copy-on-write.

7.5 Completed code

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int status;
    pid_t pid;
    printf("shell: running job 1\n");
    fflush(stdout);
    pid = fork();
    if (pid == 0) {
        execlp("expr", "expr", "6", "*", "7", (char *)NULL);
        exit(127);
    }
    waitpid(pid, &status, 0);
    printf("job 1 exit status %d\n", WEXITSTATUS(status));

    printf("shell: running job 2\n");
    fflush(stdout);
    pid = fork();
    if (pid == 0) _exit(3);
    waitpid(pid, &status, 0);
    printf("job 2 exit status %d\n", WEXITSTATUS(status));
    return 0;
}
```

Compile and run: **gcc -std=c11 -Wall -Wextra t7.c -o t7 && ./t7**

Output obtained (same as the expected output):

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

7.6 Why stdout is flushed before fork(), and the duplicated line

When stdout is connected to a terminal it is line-buffered, but when it is redirected to a file or pipe it is fully buffered. A buffered line that has not yet been written is part of the process's

memory, so fork() copies it into the child as well. If the buffer is not flushed before fork(), both the parent and the child eventually flush the same text and it is printed twice. fflush(stdout) before each fork() empties the buffer so the child starts with an empty buffer.

Experiment: the second fflush(stdout) was removed, _exit(3) changed to exit(3), and the output redirected (**./t7b > out.txt**). The file contained:

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Explanation: with redirection the text "job 1 exit status 0" and "shell: running job 2" were still in the parent's buffer when the second fork() happened. The child copied the buffer and, because exit(3) (unlike _exit(3)) flushes stdio buffers, the child printed the copied text before terminating; the parent then printed it again when it exited. Hence "shell: running job 2" (and the preceding line) appear twice.

7.7 Alternative concept: posix_spawn() or threads

Criterion	fork + exec	posix_spawn()	Threads
Time to start 1,000 commands	About 60 ms measured for 1,000 fork+wait (child exits at once); proportional to the page-table copy of the parent	Similar or lower (uses vfork/clone-vm internally); avoids copying the parent's page tables	About 11 ms measured for 1,000 create+join, but threads cannot run a new program
Memory copied / shared	Address space copied lazily with copy-on-write; replaced at exec	Child is built directly with the new image; almost nothing copied	All memory shared; only a stack (for example 8 MB virtual) per thread
Isolation	Full – separate address spaces; a crash in one command does not affect the shell	Full – a separate process	None – an error or bad pointer in one thread can corrupt or kill the entire program

Evaluation: for a database-server shell, isolation and fault containment are more important than saving milliseconds; fork+exec (or posix_spawn, which is a faster way to do the same thing) is correct, whereas threads are unsuitable for running independent commands.

TASK 8 – Evaluate CPU States and Protection

> **Task 8 (K5 – Evaluate)** **[3 Marks]**
>
> Evaluate how the hardware protects the hospital system: user and kernel mode, privileged instructions and the mode bit; interrupts, traps and the timer interrupt; what would happen if any program could disable interrupts; how the hardware stops one program reading another's memory.
>
> **Alternative Concept:** If the monitors used polling instead of interrupts, evaluate the worst-case delay (polling interval vs interrupt latency) and CPU time and power wasted.

Answer:

8.1 User mode, kernel mode and privileged instructions

The CPU has a **mode bit**: 1 = user mode, 0 = kernel (supervisor) mode. Ordinary programs run in user mode and can execute only non-privileged instructions. **Privileged instructions** – I/O

instructions, enabling/disabling interrupts, setting the timer, loading the memory-management registers, halting the CPU, changing the mode bit – work only in kernel mode; an attempt in user mode causes a trap to the OS. A program enters kernel mode only through a system call, interrupt or trap, and the OS resets the mode bit to user mode before returning control to the program.

8.2 Interrupts, traps and the timer

- **Interrupt:** a hardware signal (sensor, disk, network, timer) that makes the CPU save its state and run an interrupt-service routine, then resume.
- **Trap (exception):** a software-generated interrupt caused by an error (divide by zero, invalid memory access) or by a system call.
- **Timer interrupt:** a hardware timer, loaded by the OS, interrupts the CPU at fixed intervals so the OS regains control; this makes pre-emptive scheduling possible and stops any program from keeping the CPU forever.

8.3 If any program could disable interrupts

A faulty or malicious program could disable interrupts and then (i) the sensor interrupt would not be serviced, so abnormal readings would not be noticed and **no alarm would be raised within 10 ms**; (ii) the timer interrupt would not occur, so the OS could never pre-empt the program and the monitor would hang; (iii) network and disk interrupts would be lost. This is why disabling interrupts is a privileged instruction.

8.3.1 Memory protection

The hardware Memory Management Unit (MMU) translates each program's logical addresses using a base/limit register pair or page table that only the kernel can change. Every access is checked: an address outside the program's own space causes a trap (segmentation fault) and the OS terminates the program. Therefore one program cannot read or modify another program's memory or the kernel's memory.

8.4 Alternative concept: polling instead of interrupts

Criterion	Polling (loop checks the sensor)	Interrupt-driven
Worst-case delay	Equal to the polling interval T plus processing time; e.g. T = 20 ms would miss the 10 ms deadline, so T must be below 10 ms	Interrupt latency only: a few microseconds (dependent on interrupt masking)
CPU time and power	The CPU keeps running and reading the sensor even when nothing changes: about 100% of the time with busy waiting, or 1/T wake-ups per second with timed polling. Higher power consumption	CPU executes code only when an event occurs and can sleep between events, saving power and battery
Scalability	Cost increases with the number of sensors (each polled in every cycle)	Cost proportional to the number of events

Polling is acceptable only when events are very frequent or the interval is very short; for rare abnormal readings on battery-backed monitors, interrupts give the faster response and lower power, so the interrupt design is retained.

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems

Task 9 (K5 – Evaluate) **[4 Marks]**

Evaluate how data moves between storage levels and devices: levels of the memory hierarchy with typical access times and sizes; where MRI images, patient records and monitor readings should be kept; programmed I/O, interrupt-driven I/O and DMA and which suits the MRI scanner; the effect of caching on record look-up.

Alternative Concept: If the MRI scanner used interrupt-driven I/O (one interrupt per word) instead of DMA, evaluate the number of interrupts and CPU time for a 500 MB image, and the memory buffers needed in each method.

Answer:

9.1 Memory hierarchy (typical values)

Level	Access time	Typical size	Volatile / note
Registers	< 1 ns	Hundreds of bytes	Volatile; inside CPU
Cache (L1/L2/L3)	1–15 ns	32 KB – tens of MB	Volatile; hardware managed
Main memory (DRAM)	50–100 ns	8 GB – TBs	Volatile
SSD	50–150 µs	100 GB – several TB	Non-volatile, no moving parts
Hard disk	5–10 ms	Several TB	Non-volatile, cheap per GB
Network storage / cloud	0.5 ms – 100 ms	Petabytes	Non-volatile, shared, remote

Each level down is about 10–1000 times slower but larger and cheaper per byte; the OS and hardware exploit locality of reference by keeping recently used data in faster levels.

9.2 Data-placement plan

Data	Where it is kept	Reason
Monitor readings (live)	Main memory/ring buffer in the monitor; periodic copy to the server database (SSD)	Needs nanosecond–microsecond access for alarm checking; long-term history needs non-volatile storage
Patient records	Database on SSD with a large DRAM cache; replicated on the cluster's shared storage; backups on hard disk/network storage	Frequent random look-ups need fast access; durability needs redundancy
MRI images (hundreds of MB each)	Processing in the imaging server's RAM; storage on large hard-disk/network storage arrays; recent scans on SSD	Too big for RAM and cache to hold the archive; sequential access suits disks; recent scans are accessed more often

9.3 I/O methods

- **Programmed I/O (polling):** the CPU repeatedly tests a device status register and copies each word itself. Simple, but the CPU is busy for the entire transfer.
- **Interrupt-driven I/O:** the device interrupts the CPU when it is ready; the CPU copies one word/byte in the service routine. The CPU can do other work between interrupts, but each word costs an interrupt.
- **DMA (direct memory access):** the CPU gives the DMA controller the memory address, the byte count and the direction; the controller moves a whole block between device and memory without the CPU and interrupts once at the end. **Best for the MRI**

scanner (large, high-speed block transfers); interrupt-driven I/O suits the keyboard and the sensors; programmed I/O only very simple devices.

9.4 Effect of caching on record look-up

Caching keeps copies of recently used records in faster memory. With average access time = h × t_cache + (1 – h) × t_disk, a hit rate of 95% with t_cache = 100 ns and t_disk = 100 µs gives about 0.95 × 0.1 + 0.05 × 100 = 5.1 µs, i.e. about 20 times faster than every look-up going to the disk (100 µs). The hospital database should therefore have a large buffer cache in RAM; because the same patients' records are usually used repeatedly during treatment, locality gives a high hit rate. Cache coherence and write-through/write-back policies are needed so that a doctor never sees an outdated record.

9.5 Alternative concept: interrupt-driven I/O for a 500 MB image

Assumptions: 500 MB = 524,288,000 bytes, a word = 4 bytes, interrupt-service cost about 2 µs per interrupt, DMA block = 4 KB.

Criterion	Interrupt-driven I/O (one interrupt per word)	DMA
Number of interrupts	524,288,000 / 4 = 131,072,000 interrupts	524,288,000 / 4096 = 128,000 (or a single interrupt when the whole transfer completes)
CPU time	131,072,000 × 2 µs ≈ 262 s of CPU time (over 4 minutes)	128,000 × 2 µs ≈ 0.26 s of CPU time (or less)
Memory buffers	Only a small word register/buffer for the transfer, but the whole 500 MB destination area is still needed	DMA descriptors and block buffers (e.g. two 1 MB buffers for double buffering) in addition to the destination area

DMA reduces interrupts by a factor of about 1000 and leaves the CPUs free to process earlier slices of the scan, so DMA is selected; the small extra buffer memory is an acceptable cost.

TASK 10 – Measure and Compare

> **Task 10 (K5 – Evaluate)** **[4 Marks]**
>
> Measure the behaviour of your programs on Linux: number of system calls for different buffer sizes (strace -c); time in user and kernel mode (time command); time to create 1,000 processes with fork() and 1,000 threads; how the measurements support or contradict earlier judgements.
>
> **Alternative Concept:** If threads were created instead of processes, evaluate the creation time for 1,000 threads vs 1,000 processes and memory used, and explain the difference using what each shares.

Answer:

Test system: Ubuntu 24.04 LTS (Linux), gcc 13.3.0, compiled with **gcc -std=c11 -Wall -Wextra -pthread**. Test: copy a 4 MB file; process/thread creation with a loop of 1,000 iterations.

10.1 System calls and time for different buffer sizes

Buffer size	read() calls (calculated)	strace -c count (10 KB test)	Real (s)	User (s)	Kernel (s)
1 byte	4,194,304	10,001 (+1 loader)	2.473	0.517	1.786
512 bytes	8,192	21	0.014	0.000	0.008

Buffer size	read() calls (calculated)	strace -c count (10 KB test)	Real (s)	User (s)	Kernel (s)
4096 bytes	1,024	4	0.007	0.000	0.004
65536 bytes	64	–	0.007	0.000	0.003

Observation: with a 1-byte buffer 77% of the 2.3 s elapsed time was kernel time (1.786 s) – the program does almost nothing but enter and leave the kernel. For 4 KB and larger buffers the kernel time falls by more than 400 times, and the improvement beyond 4 KB is small.

10.2 Process and thread creation (1,000 each, create and wait/join)

Operation	Measured time	Per creation
fork() + waitpid() – 1,000 processes	0.0596 s	about 60 µs
pthread_create() + pthread_join() – 1,000 threads	0.0108 s	about 11 µs

Thread creation was about **5.5 times faster** than process creation (0.0596 / 0.0108).

10.3 Comparison with earlier judgements

Earlier judgement	Measurement	Conclusion
A system call is far costlier than a function call (Task 6)	1,000 times more calls (1 byte vs 4 KB) made the copy about 340 times slower	Supported
Buffering/large buffers reduce calls proportionally (n / b)	Calls fall from 4,194,304 to 1,024 to 64 as b rises	Supported; time gains saturate after 4–64 KB because disk/page-cache speed then dominates (partly different from the simple "time ∝ calls" expectation)
Threads are cheaper than processes (Task 7)	Threads 5.5x faster to create	Supported
Library buffering reduces calls (Task 5)	10,000 fputc() calls made only 3 write() calls (strace)	Supported

10.4 Alternative concept: threads instead of processes

Creation time was 0.0108 s for 1,000 threads and 0.0596 s for 1,000 processes. The difference is explained by what each shares: a thread shares the code, data, heap and open files of its process and needs only a new stack and thread control block (a stack is allocated lazily; for example 8 MB of virtual address space but only a few KB resident), so the kernel does not copy any page tables. A process has its own address space, so fork() must create and copy the page tables (copy-on-write), the process-table entry and the file-descriptor table, and the child's later writes cause page copies. Therefore a process uses more memory (tens of KB to MB) and takes longer to create, but offers isolation; a thread is lighter but a fault in one thread can affect all threads.

TASK 11 – Recommend and Defend the Operating-System Plan

> **Task 11 (K5 – Evaluate)** **[4 Marks]**
>
> Recommend the complete OS plan: type and structure of OS for each environment; services, protection and I/O methods; limitations and risks; suitability for a group of three hospitals — based on evidence from your own analysis and measurements.

Alternative Concept: If the whole system were moved to virtual machines on two large servers instead of separate physical machines, evaluate the effect on the monitors' response time, memory and processor overhead and reliability.

Answer:

11.1 Recommended plan

Environment	OS type	Structure	Main services/ protection	I/O method	Example OS
Bedside monitor	Hard real-time	Microkernel	Scheduler, interrupts, alarm, small network stack; memory protection	Interrupt-driven	QNX / FreeRTOS
Nurse tablet	Handheld (soft real-time)	Layered (Android) / hybrid	Touch UI, power management, app sandbox, encryption, remote wipe	Interrupt-driven; DMA for network	Android/ iOS-type
Doctor workstation	Desktop (time-sharing)	Hybrid/ modular	GUI, authentication, file protection	Interrupt-driven, DMA for disk	Windows/ Linux desktop
Database server	Clustered server OS with fail-over	Monolithic with modules	Journalled file system, ACLs, encryption, audit, fail-over	DMA with large cache	Linux cluster
Imaging server	Symmetric multiprocessing	Monolithic with modules	Multi-threading, memory management, high-throughput I/O	DMA	Linux SMP
Partner link	Distributed services	Modular	Authentication, TLS encryption, replication	DMA/ interrupt network	Linux gateway

11.2 Evidence from the analysis

- The 10 ms alarm cannot be guaranteed by a general-purpose OS (Task 1) and interrupts give microsecond latency while polling needs an interval below 10 ms (Task 8) – so an interrupt-driven RTOS on the monitors.
- Microkernel containment of a faulty driver outweighs its few extra microseconds of IPC (Task 3).
- Measurements: 1-byte copying was about 340 times slower than 4 KB copying, so system calls must be minimised – use 64 KB buffers and DMA (Tasks 6, 9, 10).
- Cluster fail-over improves availability from about 99.9% to about 99.9999% (Task 2); DMA reduces CPU time for a 500 MB scan from about 262 s to about 0.26 s (Task 9).

11.3 Limitations and risks

- More OS families means higher administration cost, staff training and patch management.
- Hard real-time guarantees depend on correct priorities and worst-case execution time analysis; a mistake can still miss deadlines.
- Fail-over takes 10–30 s and shared storage remains a possible single point of failure unless it is replicated.
- Lost tablets and insider access create privacy risk; encryption, auditing and remote wipe are required.
- Measurements were made on a test machine; hospital hardware must be re-measured.

11.4 Suitability for three hospitals

The plan scales: monitors and tablets scale by adding devices; the database cluster is extended by adding nodes and replicating records between hospitals (distributed services); a central management and update system with a staged roll-out is needed, and the partner link must use authenticated, encrypted replication. Because only the central services are shared, the plan remains suitable for the group, though the network between hospitals becomes critical and local copies must continue to work if it fails.

11.5 Alternative concept: virtual machines on two large servers

Criterion	Virtual machines on two servers	Separate physical machines
Response time of monitors	The hypervisor adds scheduling delay and jitter; the 10 ms guarantee is difficult unless real-time VMs and CPU pinning are used. Monitors should remain physical devices at the bedside	Direct hardware control; guaranteed latency
Memory and CPU overhead	About 2–10% CPU overhead plus memory for each guest OS and the hypervisor (e.g. 1–2 GB per VM)	No virtualisation overhead; hardware is often under-used
Reliability	Consolidation: a host failure takes down many VMs at once, but live migration and HA clustering between the two hosts can restart VMs in seconds; easier backups and snapshots	Failure is contained to one machine; more hardware to maintain

Evaluation: virtualisation is good for the database, imaging and workstation services (better utilisation and easier management, with HA between the two servers), but not for the bedside monitors; a hybrid of physical real-time monitors and virtualised back-end servers is recommended.

TASK 12 – Re-evaluate and Adapt the Design (Day-4 Design Challenge)

Task 12 (K5 – Evaluate) **[8 Marks]**

Changed requirement: the hospital adds remote monitoring devices in patients' homes that send readings over the internet, and moves patient records to a cloud service shared with the partner hospitals. Re-evaluate the original design: RETAIN, MODIFY or REDESIGN.

A. Impact analysis (2) · B. Evaluation of the original design (2) · C. Adaptation or redesign (2) · D. Evidence-based defence (2)

Answer:

A. Impact Analysis (2 Marks)

Area	Impact	Affected OS type/service/protection
Home monitors	New environment: battery or mains, unreliable home internet, no IT staff, patient-owned network	New embedded real-time OS; secure remote updates; power management
Cloud records	Records leave the hospital's own data centre; shared by three organisations	Distributed OS services, replication, access control, accounting across organisations
Security & protection	Attack surface moves from the private LAN to the internet; data in transit and in the cloud	Authentication, TLS encryption, certificates, secure boot, audit logs, protection between tenants

Area	Impact	Affected OS type/service/protection
Response time	Alarm must still be local on the device, as the internet adds 20–200 ms and may fail	Local hard real-time alarm; network only for reporting
Reliability	Dependence on the internet and cloud provider	Offline operation and store-and-forward buffering; fail-over for the cloud service
Existing parts	Database cluster, imaging server and workstations are largely unaffected locally; tablets use the cloud API	Clustered database becomes a local cache/replica or is replaced by cloud storage

B. Evaluation of the Original Design (2 Marks)

- **Response time:** the monitor design (hard real-time, interrupts, microkernel) remains valid for the alarm itself, but the design assumed a local, reliable network; a cloud alarm path cannot guarantee 10 ms.
- **Reliability:** the on-premises cluster protects against node failure but not against loss of the internet connection or cloud outage; home devices have no redundancy.
- **Security:** the original protection (file permissions, memory protection, LAN isolation) is not enough for devices in homes and shared cloud records; it lacks device authentication, end-to-end encryption and multi-organisation access control.
- **Conclusion:** the original design can not support the new requirements unchanged, but its core choices (RTOS for monitors, clustered database, handheld OS for tablets) are sound and reusable.

C. Adaptation or Redesign (2 Marks)

Decision: **MODIFY** (retain the hospital design; extend it with new components):

- **Home devices:** an embedded hard real-time OS (FreeRTOS/Zephyr-type microkernel) with local alarm processing, a small TCP/IP stack with TLS, secure boot, signed over-the-air updates, and store-and-forward buffering when the internet is down; low-power interrupt-driven design.
- **Cloud records:** a distributed system – replicated, encrypted database with role-based access, audit logs and consent control for each hospital; the local clustered database becomes the primary on-site replica so that local care continues if the cloud is not reachable.
- **Gateway services:** secure API gateway with authentication (certificates/OAuth), rate limiting and intrusion detection between home devices, hospital and cloud.
- **Existing devices:** bedside monitors, imaging server and workstations are unchanged; tablets connect through the gateway.

D. Evidence-Based Defence (2 Marks)

Why modify and not retain: Task 1 showed that the alarm deadline can only be guaranteed by a local hard real-time OS (Task 8: interrupt latency is microseconds, while network round-trip times are tens or hundreds of milliseconds) – so the alarm stays on the device. **Why not completely redesign:** the measurements and analysis of Tasks 3, 6, 9 and 10 (microkernel containment, DMA, buffer sizes and the cluster's 99.9999% availability) apply unchanged to the hospital's own devices and servers; redesigning everything would waste working, justified

components and increase risk. **Trade-offs:** (i) cost and complexity increase (new device OS, key management, cloud contracts); (ii) latency of the shared records increases but it is acceptable for non-alarm data; (iii) security improves through encryption and audit but adds CPU and battery usage on the home devices; (iv) a local replica costs more storage but gives availability when the internet fails. **Final decision:** modify the original plan by adding an embedded RTOS for home devices, a secure gateway and a replicated, access-controlled cloud record system while keeping the local clustered database as the on-site replica.